

Computer Arkitektur og Operativ Systemer

Assembler 1

Kursusgang 3
Brian Nielsen

*Credits to
Randy Bryant & Dave O'Hallaron (CMU)*

QUIZZ fra LEKTION 1: “Ord og Endianess, side 2!

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

In a computer using LITTLE endian, what is the value of the byte stored in address 0x514 ? 0x..

Drag answer here

In a computer using BIG endian, what is the address containing the least significant byte 0x7B? 0x...

Drag answer here

In a computer using LITTLE endian, what is the address containing the most significant byte 0x1A? 0x...

Drag answer here

In a computer using BIG endian, what is the value of the byte stored in address 0x514 ? 0x..

Drag answer here

In a computer using LITTLE endian, what is the address containing the least significant byte 0x7B? 0x...

Drag answer here

In a computer using LITTLE endian, a 16-bit word starts at address 0x514 ? What hex-value is stored there? 0x..

Drag answer here

In a computer using BIG endians, what is the address containing the most significant byte 0xA1? 0x...

Drag answer here

510

517

56f4

f4

56

bc

f456

5a0

d5

QUIZZ

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

In a computer using LITTLE endian, what is the value of the byte stored in address 0x514 ? 0x..

In a computer using BIG endian, what is the address containing the least significant byte 0x7B? 0x...

In a computer using LITTLE endian, what is the address containing the most significant byte 0x1A? 0x...

In a computer using BIG endian, what is the value of the byte stored in address 0x514 ? 0x..

In a computer using LITTLE endian, what is the address containing the least significant byte 0x7B? 0x...

In a computer using LITTLE endian, a 16-bit word starts at address 0x514 ? What hex-value is stored there? 0x..

In a computer using BIG endians, what is the address containing the most significant byte 0xA1? 0x...

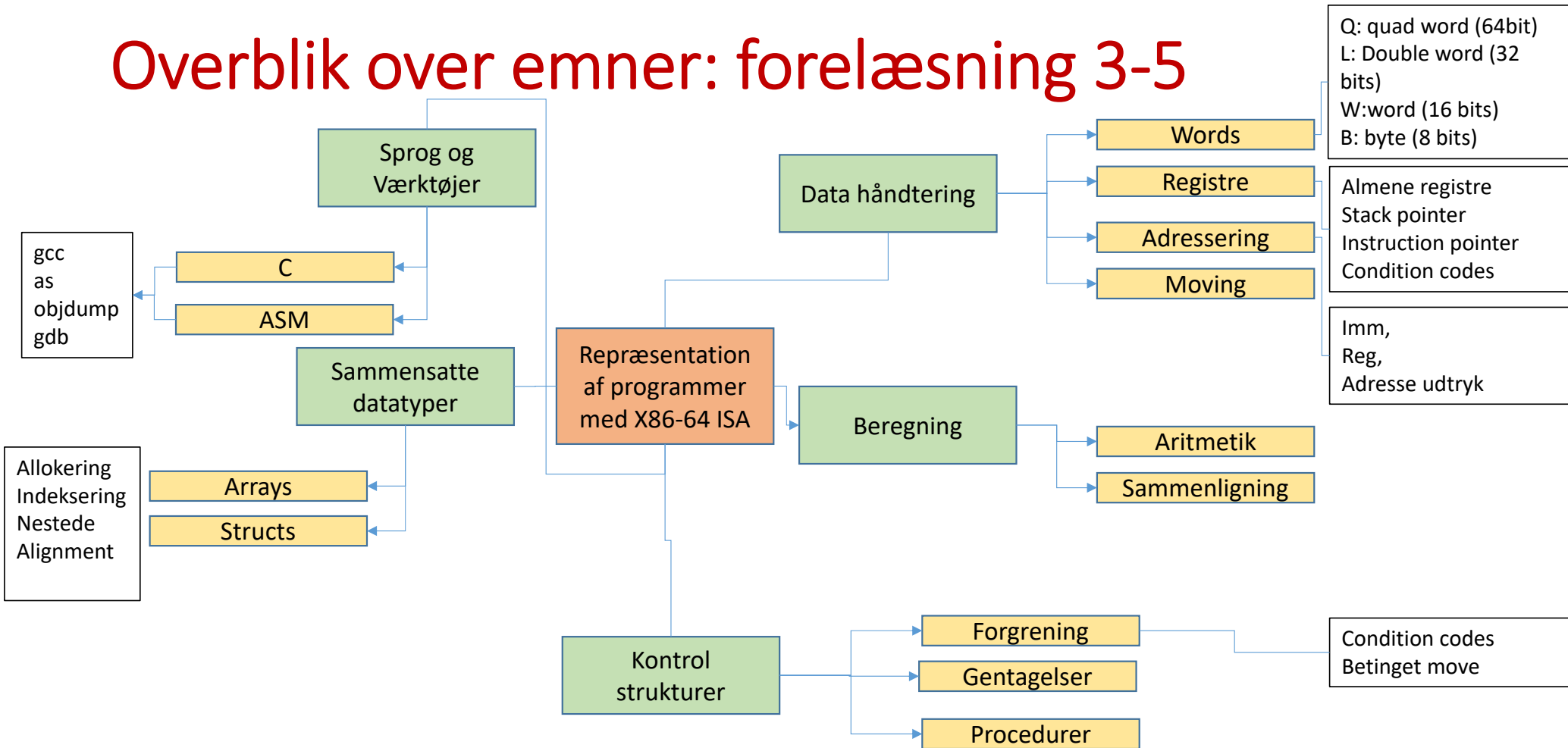


Little vs Big End.



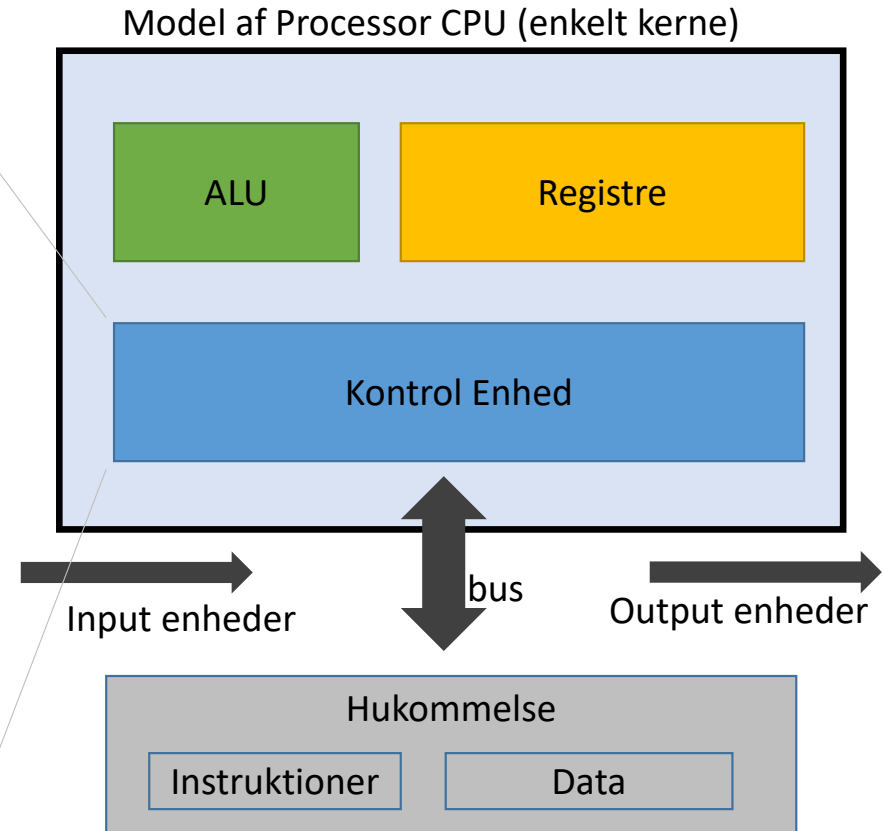
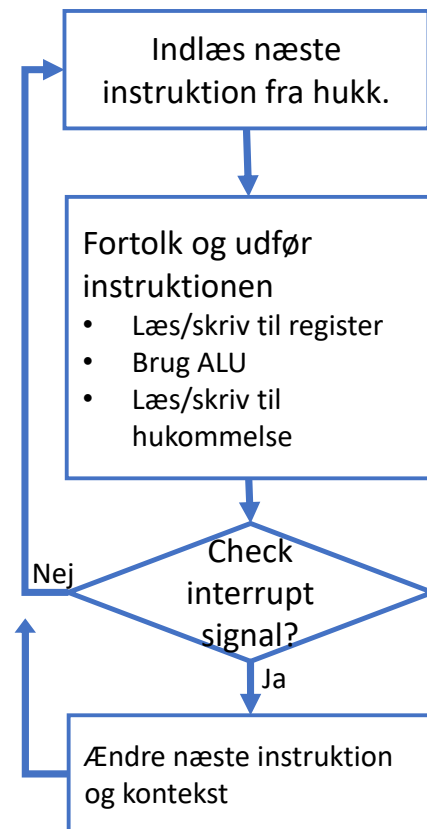
- Har vi heldig vis kun brug for i få sjældne situationer
- Typisk,
 - i debug situation hvor en adresse eller heltals-konstant manuelt udlæses fra hukommelsen
 - Debugger værktøj viser tallene "normalt"
 - En adresse fremgår af maskinkoden (altså bytekoden!)
 - assembler håndterer normalt konvertering
- X86 er little End-ian
- ARM er bi-Endian, normalt konfigureret til little-End.

Overblik over emner: forelæsning 3-5



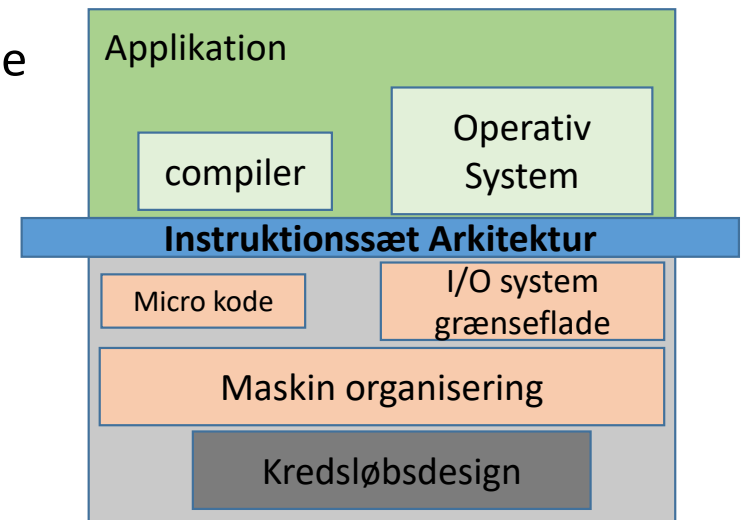
Model af CPU (Von Neuman)

- Data og program i samme hukommelse
- Program Instruktions er bare sekvenser af bits ("data") i huk.
- "Fortolker løkke" ("i-fetch-loop")
- Von Neuman flaskehals
- Modsat "Harvard" arkitektur med separate data og instruktions hukommelse og busser
- Men moderne processorer har separate L1 caches for instruktioner og data



Hvad er en ISA ?

- **ISA-Arkitektur:** (instruction set architecture) De dele af processorens design, som man skal forstå for at kunne læse/skrive assembler og maskinkode.
 - Instruktioner? Ordlængde? Registre? Indkodning?, mv.
- X86-64? ARM? RISC-V?
 - ARM: Se evt. [\[DIS\] Kap. 9](#)
- CISC vs RISC
- Hvordan ser programmer ud på ISA-niveau?



Those who say *"I understand the general principles, I don't want to bother with the details"* are eluding themselves!
(CSPP3, p201)

Assembler

```
push    %rbx
mov     %rdx,%rbx
callq   400517 <plus>
mov     %rax,(%rbx)
pop     %rbx
retq
```

Maskin-kode i hukommelsen

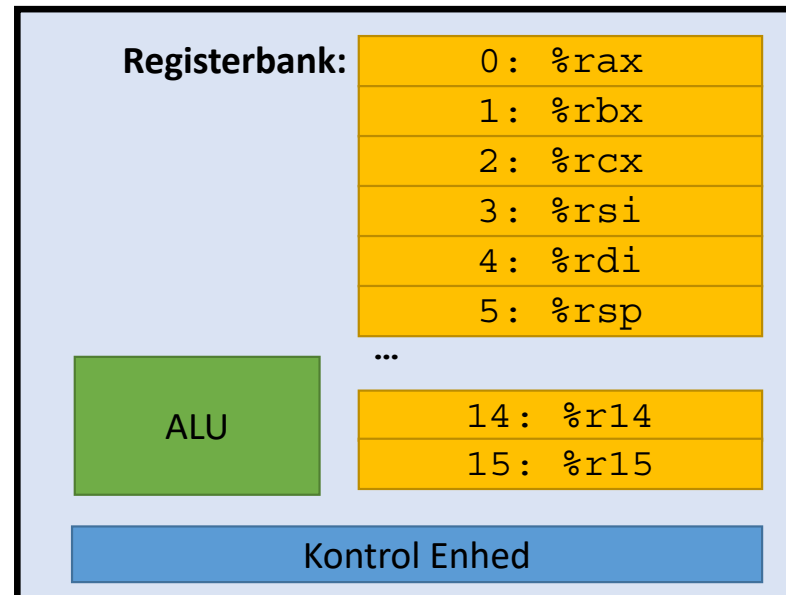
```
40051c: 53
40051d: 48 89 d3
400520: e8 f2 ff ff ff
400525: 48 89 03
400528: 5b
400529: c3
```

Lidt Notation

VIGTIG!

- En processor (kerne) har et lille internt lager, register-bank **R**:
 - Hvert register har et navn og tilhørende unikt nummer
 - Kan hvert gemme et ord
 - På x86-64: 16 registre (%rax,...), 8 bytes (w=64)
 - Array notation **R[register id]**: værdien som er gemt i registerbanken i register med angivne id, fx **R[0]** el. **R[%rax]**
- Primær-hukommelse **M** (primary memory / arbejdshukommelse / DRAM) gemmer et meget stort antal bytes
 - Hver position kaldes en **adresse** og udpeger én byte
 - Kan opfattes som et meget stort array; adresse bruges som indeks
 - Mange adresser => vi bruger typisk de mere kompakte hexa-decimale tal for adresser
 - M₁[0x089aa]**: udlæser én byte fra adressen 0x089aa
 - M₄[0x089aa]**: udlæser det 4 byte ord, der starter i adressen 0x089aa, dvs. bytes fra 0x089aa, 0x089ab, 0x089ac, 0x089ad
 - M₈[0x089aa]**: udlæser det 8 byte ord, der starter i adressen 0x089aa
- Transport af data mellem processor (kerne) og hukommelsen
 - R[%rsi]=M₈[0x89aa]**: udlæser 8 byte ord fra hukommelsen og gemmer det i registerbanken i register %rsi.
 - R[%rsi]**: $\leftarrow$ M₈[0x89aa]: ditto
 - "LOAD"
 - M₈[0x89aa]=R[%rsi]**: udlæser indholdet af %rsi registeret og gemmer det i hukommelsen med start i adresse 0x89aa
 - M₈[0x89aa]**: $\leftarrow$ R[%rsi]: ditto
 - "STORE"

Model af Processor CPU (enkelt kerne)



Hukommelse	
Adresse	Indhold
00000000	1-byte
00000001	1-byte
00000002	1-byte
00000003	1-byte
00000004	1-byte
00000005	1-byte
...	
ffffffff	1-byte

Assembler?!!

```
0000000000000000 <.text>:
0:  48 8b 1e          mov    (%rsi),%rbx
3:  48 8b 07          mov    (%rdi),%rax
6:  48 01 d8          add    %rbx,%rax
9:  48 89 07          mov    %rax,(%rdi)
```

Komplexitet:

- Hver instruktion gør kun en lille bid af arbejdet
- Der skal mange instruktioner til
- De præcise rigtige skal bruges
- De skal sættes rigtigt sammen
- Syntaks ikke intuitiv
- Semantik (effekt af instruktioner) detaljeret
= specifik ændring i maskinens tilstand
- “Regularitet” er ikke et vigtigt
“sprog-design” kriterie i asm 😊

- `long * a; long * b;`
 - `a` gemt i hukommelsen i adressen udpeget af register `%rdi`
 - `b` gemt i hukommelsen i adressen udpeget af register `%rsi`
- `*a = *a + *b //a=a+b;`
- Assembler som Prosa
 - 0: load quad word from memory address given in register `%rsi` and store value in register `%rbx` //load `b`
 - 3: load quad word from memory address given in register `%rdi` and store value in register `%rax` //load `a`
 - 6: add quad words in registers `%rax` and `%rbx` and store result in `%rax` //add `a+b`
 - 9: Store quadword in register `%rax` into memory address given by register `%rdi`. //store `a+b`.
- C-“skridt for skridt”
 - 0: `long tmp1=*a;`
 - 3: `long tmp2=*b;`
 - 6: `tmp2 +=tmp1;`
 - 9: `*a=tmp2;`
- Med præcise data-flytninger
 - 0: $R[\%rbx] = M_8[R[\%rsi]]$
 - 3: $R[\%rax] = M_8[R[\%rdi]]$
 - 9: $R[\%rax] = R[\%rax] + R[\%rbx]$
 - 6: $M_8[R[\%rdi]] = R[\%rax]$

Hvad er registre?

- Et lille stykke lager internt i en processer-kerne
 - Lagring af midlertidige variable og resultater
 - Rasende hurtig adgang, men relativt få styks
 - I X86-64: 16 almene registre a 64 bits ord! Fx **%rdx**
 - "Virtuelle registre", fysisk delmængder af 64 bit registrene
- Modsæt: primær "hukommelsen"
 - maskinens RAM, Giga-bytes, externt til processoren, meget langsom ifht. processoren.



Hvis %rdx indeholder ordet 0xFFFFFFFFEEEEDDCC, så har

- %edx: 0xEEEEEDDCC
- %dx: 0xDDCC
- %dh: 0xDD
- %dl: 0xCC

movb \$0x11,%dh, vil %rdx indeholde 0xFFFFFFFFEEEE11CC

movl \$0x11,%edx,vil %rdx indeholde 0x0000000000000011

!64-bit finte (aside p220)

Hvordan laver vi variable, assignments, og udtryk i ASM?

- *Variable* findes kun som "ord", som er gemt i hukommelsen eller et register
 - Identificeres vha. den adresse ordet starter på ("pointer"), eller
 - vha. registernavn
 - Om og hvornår variable gemmes registre bestemmes af
 - Compiler (eller programmørens) "register allokeringsstragi" (hyppigt anvendte i registre)
 - Instruktions kræver oftest at operander befinder sig i registre
 - Behov for at skabe en pointer
 - Konventioner for parametre til funktions-kald
- **move** *src,dst* instruktionerne bruges til at
 - Kopiere et ord fra hukommelse til register (load), eller tilbage (store), eller
 - Kopiere et ord imellem 2 registre
 - Ord findes i forskellige størrelse 8,16,32,64 bits
- **lea** *src,dst* "*load effective address*"
 - Instruktionerne bruges til at *beregne* en adresse på et ord
 - Men en adresse er bare et (unsigned) heltal.

Fra C-til Assembler og tilbage

- Bogen anvender en vis metodik som kan bruges til lettere at forstå
 1. Hvordan compileren laver assembler
 2. Forstå assembler som C-kode.

Idag: data i register kan opfattes som temporære variable i C

```
long t, long *dest;  
...  
*dest = t;  
long t2=t;
```

compilering

“de-compilering”

```
#dest in %rbx  
#t in %rax  
movq %rax, (%rbx)  
movq %rax, %rsi
```

```
long absdiff  
(long x, long y)  
{  
    long result;  
    if (x > y)  
        result = x-y;  
    else  
        result = y-x;  
    return result;  
}
```

```
long absdiff_j  
(long x, long y)  
{  
    long result;  
    long ntest = x <= y;  
    if (ntest) goto Else;  
    result = x-y;  
    goto Done;  
Else:  
    result = y-x;  
Done:  
    return result;  
}
```

```
##%rdi=x, %rsi=y  
##%rax=result  
absdiff:  
    cmpq    %rsi, %rdi # x:y  
    jle     .L4  
    movq    %rdi, %rax  
    subq    %rsi, %rax  
    ret  
.L4:      # x <= y  
    movq    %rsi, %rax  
    subq    %rdi, %rax  
    ret
```

Fra C-til Assembler og tilbage

- Bogen anvender en vis metodik som kan bruges til lettere at forstå
 1. Hvordan compileren laver assembler
 2. Forstå assembler som C-kode.

parantes () angiver et **adresse-udtryk!**

- Del af instruktionsformatet / syntax
- Ikke operator præcedens el. parameterliste!

```
long t, long *dest;
...
*dest = t;
long t2=t;
```

compilering

“de-compilering”

Idag: data i register kan opfattes som temporære variable i C

```
#dest in %rbx
#t in %rax
movq %rax, (%rbx)
movq %rax, %rsi
```

```
long absdiff
(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
long absdiff_j
(long x, long y)
{
    long result;
    long ntest = x <= y;
    if (ntest) goto Else;
    result = x-y;
    goto Done;
Else:
    result = y-x;
Done:
    return result;
}
```

```
##rdi=x, %rsi=y
##rax=result
absdiff:
    cmpq    %rsi, %rdi # x:y
    jle     .L4
    movq    %rdi, %rax
    subq    %rsi, %rax
    ret
.L4:      # x <= y
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

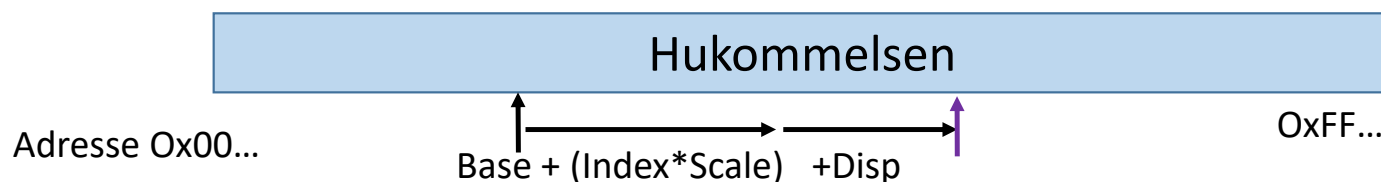


Fuldstændige adresseudtryk

- Mange instruktioner kan tage operander fra hukommelsen angivet ved et adresseudtryk:

$D(R_b, R_i, S)$ betyder: **Effektiv addr = $\text{Reg}[R_b] + S * \text{Reg}[R_i] + D$**

- D: konstant, forskydning ("displacement") 1, 2, eller 4 bytes lang
- R_b : Basis register: Ethvert af de 16 almene heltals registre
- R_i : Index register: Ethvert, undtagen `%rsp`
- S: Skaleringsfaktor: 1, 2, 4, or 8



- Special Tilfælde

	Effektive adresse	
(R_b)	$\text{Reg}[R_b]$	//Disp=0, S=0
(R_b, R_i)	$\text{Reg}[R_b] + \text{Reg}[R_i]$	//Disp=0, S=1
$D(R_b, R_i)$	$\text{Reg}[R_b] + \text{Reg}[R_i] + D$	//S=1
(R_b, R_i, S)	$\text{Reg}[R_b] + S * \text{Reg}[R_i]$	//Disp=0

Movq Src, Dst

Leaq Src, Dst

rsi=0x1000

rdi=0x100

Start Adresse	Indhold (qword)
0x100	0x12
0x1FF	0xAB
0x1000	0x42
0x1100	0xCD
0x18FF	0x99

Eksempel	Beregning af "Effektiv adresse"	Effekt Eksempel
movq \$0xFF(%rsi,%rdi,8), %rax	Format: $D(R_b, R_i, S)$ $EA = R[R_b] + S * R[R_i] + D$ $= R[R_b] + 8 * R[R_i] + 0xFF$ $= 0x1000 + 8 * 0x100 + 0xFF = 0x18FF$	%rax = $M_8[EA] = M_8[0x18FF] = 0x99$ UDLÆSNING FRA MEM!
leaq \$0xFF(%rsi,%rdi,8), %rax		%rax = EA = 0x18FF KUN ADRESSEBEREGNING
movq (%rsi,%rdi), %rax	$S=1, D=0$ Format: $0(R_b, R_i, 1)$ $EA = R[R_b] + 1 * R[R_i] + 0$ $= 0x1000 + 0x100 = 0x1100$	%rax = $M_8[EA] = M_8[0x1100] = 0xCD$
leaq (%rsi,%rdi), %rax		%rax = EA = 0x1100
movq 0xFF(,%rdi), %rax	$R_b=0, S=1$ Format: $0xFF(0, R_i, 1)$ $EA = R[R_b] + 1 * R[R_i] + 0$ $= 0x100 + 0xFF = 0x1FF$	%rax = $M_8[EA] = M_8[0x1FF] = 0xAB$
leaq 0xFF(,%rdi), %rax		%rax = EA = 0x1FF
movq (%rsi), %rax	$R_i=0, S=1, D=0$ Format: $0(R_b, 0, 1)$ $EA = R[R_b] + 1 * 0 + 0 = 0x1000$	%rax = $M_8[EA] = M_8[0x1000] = 0x42$
leaq (%rsi), %rax		%rax = EA = 0x1000
movq %rsi, %rax	Ikke adresseudtryk: ingen adresse beregning;	%rax = R[%rsi] = 0x1000
leaq %rsi, %rax		ULOVLIG

Pop-Quizz (Addressering)

Hvilken af følgende x86-64 instruktioner beregner korrekt $\text{\%rax} = 9 * \text{\%rdi}$?

- a. `movq (,%rdi,9), %rax`
- b. `leaq (,%rdi,9), %rax`
- c. `leaq (%rdi,%rdi,8), %rax`
- d. `movq (%rdi,%rdi,8), %rax`

Addresering og Data flytning 1+2

Lad

`%rdx=0xf000` og `%rcx= 0x0100`

Betragt nedenstående instruktionen:

`movq 0xa(%rdx,%rcx,8), %rax`

Fra hvilken effektiv adresse udlæses værdien som skal gemmes i `%rax`? 0x...

Answer:

Lad

`%rdx=0xf000` og `%rcx= 0x0100`

Betragt nedenstående instruktionen:

`leaq 0xa(%rdx,%rcx,8), %rax`

Hvilken værdien gemmes i `%rax`? 0x...

Answer:

Addresering og Data flytning 3

Betragt nedenstående tilstand for registre og hukommelsen før og efter en sekvens af assembler-instruktioner er kørt. Instruktionen afvikles i rækkefølgen spørgsmålene kommer i.

Hvilken sekvens af instruktioner resulterer i efter-tilstanden?

Før

Registre

%rdi	0x520
%rsi	0x500
%rax	
%rdx	

Hukommelsen

123	Adresse 0x520
	0x518
	0x510
	0x508
456	0x500

Efter

Registre

%rdi	0x500
%rsi	0x500
%rax	0x123
%rdx	

Hukommelsen

123	Adresse 0x520
	0x518
	0x510
123	0x508
456	0x500

Hvilken instruktion vil give %rax værdien 123?

Hvilken instruktion vil skrive 0x500 til %rdi registeret?

Hvilken instruktion vil skrive 0x123 til hukommelsesadresse 0x508?

Hvordan laver vi aritmetik?

- En række dedikerede instruktioner

`add` *Src, Dest* `Dest = Dest + Src`

...

`sar` *Src, Dest* `Dest = Dest >> Src` *//Aritmetisk højre skifte,...*

- **lea** bruges også til at lave simple regne-stykker i én instruktion
- Shift instruktionerne anvendes som en form for billig multiplikation
- Komplekse udtryk brydes op i mange instruktioner, bruger mange registre (evt. hukommelsen) til at gemme mellem resultater.

Aritmetiske udtryk i assembler

Betragt afviklingen af nedenstående assembler

```
imul  %rdi,%rdi
imul  %rsi,%rsi
add   %rsi,%rdi
lea   (%rdi,%rdi,1),%rax
```

Hvilket udtryk beregner det (resultat i %rax) når programmet startes med %rdi havende startværdi x, og %rsi havende startværdi y?

- a. $(2*x+2*y) + x+y+1$
- b. $2*(2*x+2*y)$
- c. $(x*x+y*y) + x+y+1$
- d. $2*(x*x+y*y)$
- e. $(2*x+2*y) + x+y$

Øvelserne

- Adressering af og indlæsning fra hukommelsen
- Kan vi læse og forstå ASM?
- De-kompilering (fra Asm til C)
 - Tildelinger og sammensatte udtryk
- Bitshift, aritmetik med bit-shift
- Challenge 2: afmontering af en binær-bombe!!

Brug Hjælpelærer assistancen! Både online og fysisk!



